# Problem Statement: Distributed Task Scheduler with Dynamic Load Management

## Context

In a distributed environment, tasks need to be scheduled across multiple worker nodes based on their availability and processing capability. The goal is to design a **Distributed Task Scheduler** that assigns tasks to the most optimal worker node while dynamically managing workloads.

---

## Functional Requirements (Must-Have Features)

1. **Worker Node Registration**
    - A worker node can register itself with the system by providing:
        - `Node ID`
        - `CPU Capacity (in cores)`
        - `Memory Capacity (in GB)`
        - `Processing Speed (Tasks per Second)`
    - The system maintains an in-memory list of all worker nodes.
2. **Task Submission**
    - A user submits a task with:
        - `Task ID`
        - `CPU Requirement (cores)`
        - `Memory Requirement (GB)`
        - `Execution Time (seconds)`
    - Tasks should be **queued if no worker is available** to execute them immediately.
3. **Dynamic Task Assignment**
    - The system assigns tasks to the **best-fit worker node** using:
        - **CPU & Memory Availability:** The worker must have enough free CPU and memory.
        - **Processing Speed:** Tasks should be assigned to the fastest available worker.
4. **Task Execution & Completion**
    - Once a task is assigned, it **blocks the required CPU & memory** on the worker node.
    - After **execution time**, the task is marked as **completed**, and the resources are released.
5. **Worker Failure & Task Reassignment**
    - Tasks assigned to failed workers should be **reassigned**.

6. **Reset Scheduler**
    - Removes all registered workers, their tasks, and time.

---

### Bonus Features (Good-to-Have Enhancements)

6. **Task Prioritization**
    - Tasks have a **priority level (High, Medium, Low)**, and **high-priority tasks should be scheduled first**.
7. **Parallel Task Execution on Workers**
    - A worker can execute **multiple tasks concurrently** as long as it has sufficient CPU and memory.
8. **Auto Scaling of Worker Nodes**
    - If tasks exceed the capacity of active workers, **new workers should be registered dynamically**.
9. **Task Timeout & Reassignment**
    - If a task **exceeds its execution time by 20%**, it is **terminated** and requeued.
10. **Task Cancellation**

- A user can cancel a task if it is **queued or running**. If running, the allocated resources should be released.

---

# Test Cases

## Must-Have Test Cases

#### Test Case 1: Register Worker Nodes

**Input:**

```
None

registerWorker({ nodeId: "W1", cpu: 4, memory: 16, speed: 5 });

registerWorker({ nodeId: "W2", cpu: 8, memory: 32, speed: 10 });

listWorkers();
```

**Expected Output:**

```
None
[
  { nodeId: "W1", cpu: 4, memory: 16, speed: 5, status: "active" },
  { nodeId: "W2", cpu: 8, memory: 32, speed: 10, status: "active" }
]
```

---

**Test Case 2: Submit Task and Assign it to Best Worker**

**Input:**

```
None
registerWorker({ nodeId: "W1", cpu: 4, memory: 16, speed: 5 });
registerWorker({ nodeId: "W2", cpu: 8, memory: 32, speed: 10 });
submitTasks([{ taskId: "T1", cpu: 2, memory: 8, executionTime: 10 },
{ taskId: "T2", cpu: 4, memory: 16, executionTime: 20 }]);
listTasks();
```

**Expected Output:**

```
None
[
  { taskId: "T1", status: "assigned", assignedTo: "W2" },
  { taskId: "T2", status: "assigned", assignedTo: "W2" }
]
```

---

### Test Case 3: Task Execution & Completion

**Input:**

```
None
registerWorker({ nodeId: "W1", cpu: 4, memory: 16, speed: 5 });
submitTasks([{ taskId: "T1", cpu: 2, memory: 8, executionTime: 10 }]);
wait(10)
listTasks();
```

**Expected Output:**

```
None
[
  { taskId: "T1", status: "completed", assignedTo: "W1" }
]
```

**Input:**

```
None
registerWorker({ nodeId: "W1", cpu: 2, memory: 8, speed: 5 });
submitTasks([{ taskId: "T1", cpu: 2, memory: 4, executionTime: 10 },
{ taskId: "T2", cpu: 2, memory: 4, executionTime: 5 }]);
listTasks();
```

**Expected Output:**

```
None
[
  { taskId: "T1", status: "assigned", assignedTo: "W1" },
  { taskId: "T2", status: "queued" }
]
```

---

**Test Case 4: Worker Failure & Task Reassignment**

**Input:**

```
None

registerWorker({ nodeId: "W1", cpu: 6, memory: 32, speed: 5 });
registerWorker({ nodeId: "W2", cpu: 8, memory: 32, speed: 10 });
submitTasks([{ taskId: "T1", cpu: 2, memory: 8, executionTime: 10 },
{ taskId: "T2", cpu: 4, memory: 16, executionTime: 20 }]);
listTasks();
simulateWorkerFailure("W2");
listTasks();
listWorkers();
```

**Expected Output:**

```
None
[
  { taskId: "T1", status: "assigned", assignedTo: "W2" },
  { taskId: "T2", status: "assigned", assignedTo: "W2" }
]
 [
  { taskId: "T1", status: "assigned", assignedTo: "W1" },
  { taskId: "T2", status: "assigned", assignedTo: "W1" }
]
```

```
[
  { nodeId: "W1", cpu: 6, memory: 32, speed: 5, status: "active" },
  { nodeId: "W2", cpu: 8, memory: 32, speed: 10, status: "inactive" }
]
```

---

## Good-to-Have Test Cases

### Test Case 5: Task Prioritization

**Input:**

```
None
registerWorker({ nodeId: "W1", cpu: 2, memory: 16, speed: 5 });
submitTasks({ taskId: "T1", cpu: 2, memory: 4, executionTime: 10, priority: "Low"
},{ taskId: "T2", cpu: 2, memory: 4, executionTime: 5, priority: "High" }]);
listTasks();
```

**Expected Output:**

```
None
[
  { taskId: "T2", status: "assigned", assignedTo: "W1" },
  { taskId: "T1", status: "queued" }
]
```

---

### Test Case 6: Auto Scaling of Worker Nodes

**Input:**

```
None

registerWorker({ nodeId: "W1", cpu: 2, memory: 8, speed: 5 });

submitTasks([{ taskId: "T1", cpu: 2, memory: 4, executionTime: 10 },{ taskId:
"T2", cpu: 2, memory: 4, executionTime: 5 }]);

listTasks()

// Simulate auto-scaling to register a new worker

autoscale()

listTasks()

listWorkers();
```

**Expected Output:**

```
None
[
  { taskId: "T1", status: "assigned", assignedTo: "W1" },
  { taskId: "T2", status: "queued" }
]
[
  { taskId: "T1", status: "assigned", assignedTo: "W1" },
  { taskId: "T2", status: "assigned", assignedTo: "W2" }
]
[
  { nodeId: "W1", cpu: 2, memory: 8, speed: 5, status: "active" },
  { nodeId: "W2", cpu: 2, memory: 4, speed: 10, status: "active" }
]
```

---

**Test Case 7: Task Timeout & Reassignment**

**Input:**

```
None
registerWorker({ nodeId: "W1", cpu: 2, memory: 16, speed: 5 });
submitTasks([{ taskId: "T1", cpu: 2, memory: 4, executionTime: 10 },{ taskId:
"T2", cpu: 2, memory: 4, executionTime: 10 }]);
listTasks();
simulateTaskTimeout("T1",13)
listTasks();
```

**Expected Output:**

```
None
[
  { taskId: "T1", status: "assigned", assignedTo: "W1" },
  { taskId: "T2", status: "queued" }
]
[
  { taskId: "T1", status: "queued" },
  { taskId: "T2", status: "assigned", assignedTo: "W1"}
]
```

---

**Test Case 8: Task Cancellation**

**Input:**

```
None
registerWorker({ nodeId: "W1", cpu: 4, memory: 16, speed: 5 });
submitTasks([{ taskId: "T1", cpu: 2, memory: 4, executionTime: 10 }]);
cancelTask("T1");
listTasks();
```

**Expected Output:**

```
None
[
  { taskId: "T1", status: "cancelled"}
]
```

---

**Test Case 9: Parallel task execution**

**Input:**

```
None
registerWorker({ nodeId: "W1", cpu: 4, memory: 16, speed: 5 });
submitTasks([{ taskId: "T1", cpu: 2, memory: 4, executionTime: 10 },{ taskId:
"T2", cpu: 2, memory: 4, executionTime: 5 }]);
wait(10);
listTasks();
```

**Expected Output:**

```
None
[
  { taskId: "T1", status: "completed", assignedTo: "W1" },
  { taskId: "T2", status: "completed", assignedTo: "W1" }
]
```

**Input:**

```
None

registerWorker({ nodeId: "W1", cpu: 4, memory: 16, speed: 5 });
registerWorker({ nodeId: "W2", cpu: 8, memory: 32, speed: 10 });
submitTasks([{ taskId: "T1", cpu: 2, memory: 8, executionTime: 10 },{ taskId:
"T2", cpu: 4, memory: 16, executionTime: 20 }]);
listTasks();
wait(12);
simulateWorkerFailure("W2");
listTasks();
listWorkers();
```

**Expected Output:**

```
None
[
  { taskId: "T1", status: "assigned", assignedTo: "W2" },
  { taskId: "T2", status: "assigned", assignedTo: "W2" }
]
 [
  { taskId: "T1", status: "completed", assignedTo: "W2" },
  { taskId: "T2", status: "assigned", assignedTo: "W1" }
]
[
  { nodeId: "W1", cpu: 4, memory: 16, speed: 5, status: "active" },
  { nodeId: "W2", cpu: 8, memory: 32, speed: 10, status: "inactive" }
```

```
]
```

---

## Evaluation Criteria

1. **Correctness** – Does the system correctly assign, execute, and complete tasks?
2. **Scalability** – Can the scheduler handle a high volume of tasks efficiently?
3. **Concurrency** – Are tasks executed in parallel without conflicts?
4. **Robustness** – Does the system handle failures and timeouts effectively?
5. **Code Readability** – Is the code modular, well-structured, and maintainable?